



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Norme di Progetto

Il nostro Way of Working

Versione	1.0.0
Stato	Approvato
Redazione	Tutto il gruppo
Verifica	Tutto il gruppo
Approvazione	Matteo Cuogo
Proprietario	ByteMe
Uso	Esterno
Destinatari	Prof. Tullio Vardanega Prof. Riccardo Cardin

Registro dei Cambiamenti

Versione	Data	Autore	Descrizione	Verifica
0.2.2	15/01/2026	Giulia Barzon	Aggiunta sezione su test automatici per qualità documentazione	Matteo Cuogo
0.2.1	11/01/2026	Giulia Barzon	Aggiunta sezione sul Piano di Qualifica	Matteo Cuogo
0.2.0	30/12/2025	Elisa Marchioro	Aggiunte subsection sui requisiti in "Analisi dei requisiti"	Matteo Cuogo
0.1.8	27/12/2025	Joseph Grant	Aggiunta sezione 'verifica' al versionamento	Elisa Marchioro
0.1.7	16/12/2025	Giulia Barzon	Gestione comunicazione durante i periodi di vacanza	Elisa Marchioro
0.1.6	5/12/2025	Elisa Marchioro	Sistemata l'introduzione	
0.1.5	5/12/2025	Elisa Marchioro	Aggiunta sezione documenti, modifica sezione pianificazione del lavoro, aggiunta sezione piano di progetto	
0.1.4	5/12/2025	Elisa Marchioro	Aggiunta analisi dei requisiti	
0.1.3	19/11/2025	Giulia Barzon	Aggiornamento regole GitHub issues e dashboard	
0.1.2	16/11/2025	Elisa Marchioro	Aggiunta descrizione dei ruoli	
0.1.1	14/11/2025	Giulia Barzon	Modifica struttura documento e aggiunta organizzazione e comunicazione	
0.1.0	5/11/2025	Elisa Marchioro	Stesura iniziale documento	

Tabella 1: Registro delle versioni del documento

Indice

1	Introduzione	4
1.1	Scopo del documento	4
1.2	Cosa sono per noi le norme di progetto?	4
2	Organizzazione e comunicazione	5
2.1	Comunicazione interna	5
2.2	Comunicazione esterna	5
2.3	Comunicazione nei periodi di vacanza/sessione	5
3	Gestione del codice e documentazione	6
3.1	Organizzazione della repository GitHub	6
3.1.1	Repository per la documentazione	6
3.1.2	Versionamento della documentazione	6
3.1.3	Verifica della documentazione	6
3.2	Utilizzo di Git e GitHub	6
3.2.1	Workflow di sviluppo	6
3.2.2	Convenzioni per i documenti	7
3.2.3	GitHub Actions per LaTeX	7
3.2.4	Gestione dei compiti tramite GitHub Issues	7
3.2.5	Dashboard per il tracking delle Issues	8
3.3	Sito Web della documentazione	8
4	Documenti	9
4.1	.github/workflows	9
4.2	Candidatura	9
4.3	RTB	9
4.4	Documenti interni	9
4.5	Verbalisti esterni	9
4.6	Verbalisti interni	9
4.7	docs	10
4.8	scripts	10
4.9	src	10
4.10	README.md	10
5	Pianificazione del lavoro	11
5.1	Divisione dei ruoli	11
5.2	Descrizione dei ruoli	11
5.2.1	Responsabile	11
5.2.2	Amministratore	11
5.2.3	Analista	11
5.2.4	Progettista	11
5.2.5	Programmatore	11
5.2.6	Verificatore	12
5.3	Sprint planning	12
6	Analisi dei Requisiti (AdR)	13
6.1	Identificazione iniziale dei requisiti	13
6.2	Analisi individuale dei casi d'uso	13
6.3	Revisione dei casi d'uso	13
6.4	Discussione dei casi d'uso con l'azienda	13
6.5	Analisi individuale dei requisiti	14

6.6	Revisione dei requisiti	14
6.7	Scrittura del documento definitivo	14
6.8	Giudizio sull'approccio adottato	14
7	Piano di Progetto (PdP)	15
7.1	Analisi dei rischi	15
7.2	Approccio adottato, identificazione dei ruoli e pianificazione delle attività	15
7.3	Riassunto di ciascuno Sprint	15
8	Piano di Qualifica (PdQ)	16
8.1	Scopo del documento	16
8.2	Contenuto e Struttura	16
8.3	Metodologia di calcolo dell'Earned Value (EV)	16
8.4	Definition of Done (DoD)	17
8.5	Strumenti e Metriche di Qualità Automatizzate	17
8.5.1	Qualità della Documentazione	17
8.5.2	Qualità del Codice	18
8.5.3	Visualizzazione dei Risultati	18

1 Introduzione

1.1 Scopo del documento

Questo documento nasce dall'esigenza di ottimizzare l'organizzazione interna del gruppo, avendo una distribuzione chiara dei compiti, una buona pianificazione delle attività e un controllo costante sui progressi durante tutta la durata del progetto. Vogliamo mantenere un ambiente di lavoro positivo e produttivo, basato sulla collaborazione, in modo da poter lavorare in maniera efficiente, risolvendo anche eventuali imprevisti riscontrati durante lo sviluppo. Crediamo nel miglioramento continuo: per questo ci impegniamo ad aggiornare e rivedere le nostre Norme di Progetto ogni volta che si renda necessario, così da garantire che restino sempre allineate alle esigenze del progetto e alla crescita del team. Le Norme di Progetto costituiscono infatti la formalizzazione del nostro Way of Working, ovvero il modo in cui intendiamo collaborare e gestire le attività nel corso del progetto.

1.2 Cosa sono per noi le norme di progetto?

Le Norme di Progetto rappresentano il nostro modo di organizzare il lavoro nel progetto. È l'insieme di pratiche, strumenti e valori che il nostro team adotterà per realizzare le proprie attività. L'obiettivo è definire delle regole che possano garantire efficienza, efficacia e collaborazione tra i membri del gruppo durante tutto il ciclo di vita del progetto.

2 Organizzazione e comunicazione

2.1 Comunicazione interna

Ogni membro del gruppo si impegna a mantenere un atteggiamento aperto e disponibile, offrendo aiuto e supporto quando necessario. Per facilitare la collaborazione utilizzeremo le seguenti modalità di comunicazione:

- **WhatsApp:** per comunicazioni rapide e organizzative
- **Discord:** per riunioni sincrone e sessioni di lavoro collaborativo
- **Google Meet:** per riunioni formali e presentazioni

2.2 Comunicazione esterna

Per le comunicazioni con proponenti e committenti:

- **Email:** per comunicazioni formali
- **Google Meet:** per incontri online con la proponente
- **Incontri in presenza:** per riunioni formali con la proponente

2.3 Comunicazione nei periodi di vacanza/sessione

Il team si impegna a rimanere operativo durante i periodi di vacanza ed esami, privilegiando la comunicazione asincrona per gli aggiornamenti. Pur consapevoli di un potenziale rallentamento, i membri concorderanno obiettivi sprint realistici e manterranno un dialogo costante. Questa evenienza è stata già considerata nell'analisi dei rischi in fase di candidatura, e sono state definite specifiche contromisure per mitigarne l'impatto.

3 Gestione del codice e documentazione

3.1 Organizzazione della repository GitHub

3.1.1 Repository per la documentazione

Per la documentazione del progetto abbiamo strutturato come segue la repository Github:

```
ByteMe/  
|-- .gitignore  
|-- 1_Candidatura/  
|-- 2_RTB/  
|-- 3_PB/  
|-- Documenti_interni/  
|-- docs/  
|-- scripts/  
|-- src/  
|-- Verballi_esterni/  
|-- Verballi_interni/
```

Ogni cartella contiene tutti i documenti relativi alle rispettive fasi del progetto. La cartella dei Documenti interni contiene le Norme di Progetto (ovvero il documento Way of working) e il Glossario. La cartella docs contiene i file relativi al sito web del gruppo, dove sono esposti i documenti in pdf. La cartella src contiene i file .tex di tutti i documenti del gruppo, che vengono convertiti in .pdf nelle cartelle corrette al momento del push sulla repository. Per rendere automatico questo processo si usa il file compileLatex.yml che si trova nella cartella .gitignore. Le cartelle relative ai verbali contengono tutti i verbali in pdf relativi alle riunioni interne del gruppo o agli incontri con la proponente.

3.1.2 Versionamento della documentazione

Per ogni documento, eccetto i verbali, vengono registrate le modifiche apportate nella tabella "Registro dei Cambiamenti". Come convenzione per il versionamento è stato adottato il formato **SEMVER** (Semantic Versioning). Ad ogni modifica, la versione del documento viene aggiornata secondo il formato **x.y.z**, dove:

- **x** (major): indica modifiche sostanziali
- **y** (minor): indica aggiunte di funzionalità
- **z** (patch): indica correzioni di errori

3.1.3 Verifica della documentazione

Ogni documento viene verificato da uno o più membri del gruppo secondo il principio della "verifica incrociata", dove i verificatori sono diversi dagli autori originali. Il documento viene considerato approvato quando:

- Tutte le verifiche sono state superate
- Il responsabile di progetto dà l'approvazione finale

3.2 Utilizzo di Git e GitHub

3.2.1 Workflow di sviluppo

Il gruppo adotta il seguente flusso di lavoro per la gestione dei documenti:

- **Sincronizzazione iniziale:** Prima di iniziare a lavorare, aggiornare sempre la repository locale con il comando:

```
git pull origin main
```

- **Modifica dei documenti:** Lavorare sui file locali seguendo le convenzioni stabilite
- **Caricamento delle modifiche:** Utilizzare la seguente sequenza di comandi per pubblicare le modifiche:

```
git add .
git commit -m "Descrizione chiara delle modifiche effettuate"
git push origin main
```

3.2.2 Convenzioni per i documenti

- Tutti i documenti devono essere salvati in formato `.tex`
- I file devono essere posizionati nella cartella `src/` all'interno della sottocartella appropriata
- Per i file multimediali (loghi, immagini) utilizzare percorsi relativi:

```
\includegraphics{../logo.png}
```

- I messaggi di commit devono essere descrittivi

3.2.3 GitHub Actions per LaTeX

Il repository è configurato con un sistema di automazione che gestisce la compilazione dei documenti:

- **File di configurazione:** `compileLatex.yml`
- **Funzionalità:** Compila automaticamente i file `.tex` in `.pdf` dopo ogni push
- **Destinazione:** I PDF generati vengono salvati nelle cartelle di destinazione appropriate
- **Vantaggi:**
 - Compilazione automatica e consistente
 - Riduzione degli errori di compilazione manuale
 - Tracciabilità delle versioni PDF

3.2.4 Gestione dei compiti tramite GitHub Issues

Per una gestione efficiente e tracciabile delle attività di progetto, il gruppo utilizza le **GitHub Issues** come strumento principale. Ogni attività viene registrata come Issue nella repository e assegnata un **numero identificativo univoco** generato automaticamente da GitHub. Le Issue vengono categorizzate tramite label specifiche e assegnate ai membri del team responsabili.

Nei verbali interni, nella sezione "Azioni da Intraprendere", viene fatto esplicito riferimento alle Issue correlate tramite il loro numero identificativo (`#numero`). Questo approccio garantisce un tracciamento completo dalle decisioni dei verbali alle attività concrete da svolgere.

3.2.5 Dashboard per il tracking delle Issues

Per organizzare meglio il flusso di lavoro, sono state create due dashboard distinte per le Issue:

- **Dashboard RTB:** dedicata alla prima fase del progetto, comprendente analisi dei requisiti, definizione dei casi d'uso, definizione architettura e scelta tecnologie
- **Dashboard PB:** riservata alla seconda fase, focalizzata sullo sviluppo del prodotto

Questa suddivisione permette di monitorare separatamente lo stato di avanzamento delle due fasi principali del progetto, garantendo che tutte le attività vengano completate entro le scadenze stabilite per ciascuna milestone.

3.3 Sito Web della documentazione

Il gruppo ha implementato un sito web per ospitare la documentazione del progetto, accessibile all'indirizzo: <https://byte25.github.io/ByteMe/>

Il sito, generato automaticamente dalla repository GitHub, offre i seguenti benefici:

- **Versionamento:** mantenimento dello storico delle versioni dei documenti
- **Accesso centralizzato:** punto unico di accesso per tutta la documentazione
- **Responsive design:** consultabile da qualsiasi dispositivo

La pubblicazione avviene tramite GitHub Pages, assicurando che la documentazione online sia sempre aggiornata all'ultima versione approvata.

4 Documenti

All'interno della repository di progetto è presente una serie di cartelle e documenti strutturati con l'obiettivo di organizzare in modo ordinato i materiali prodotti dal gruppo. Di seguito si riporta una descrizione delle principali sezioni e del loro contenuto.

4.1 .github/workflows

Questa cartella contiene gli script di automazione utilizzati per la compilazione automatica dei documenti LaTeX tramite GitHub Actions. Contiene il file `compileLatex.yml`.

4.2 Candidatura

Questa cartella racchiude tutti i file pdf relativi alla fase di candidatura del gruppo al progetto. Include la lettera di presentazione, il documento sul preventivo dei costi e dell'impegno orari e la valutazione di tutti i capitolati.

4.3 RTB

Questa cartella contiene tutti i documenti pdf relativi alla RTB (Requirements and Technology Baseline), ovvero raccoglie tutto ciò che costituisce la prima baseline del progetto. Sono presenti:

- **Analisi dei requisiti:** questo documento descrive l'analisi dei requisiti per il progetto "Second Brain". Vengono quindi descritti i casi d'uso, e per ciascuno, i rispettivi requisiti funzionali o non funzionali.
- **Piano di progetto:** questo documento ha lo scopo di definire la pianificazione, l'organizzazione e il controllo delle attività del progetto Second Brain. Viene quindi fornita una sintesi delle attività, dei risultati e dei progressi conseguiti in ogni Sprint.

4.4 Documenti interni

Questa cartella contiene i documenti pdf redatti dal gruppo per l'organizzazione interna delle attività. Sono presenti:

- **Glossario:** questo documento ha lo scopo di raccogliere i termini utilizzati nella documentazione del gruppo ByteMe durante il progetto.
- **Norme di progetto:** questo documento stabilisce regole, procedure e strumenti per organizzare il lavoro del gruppo. Include anche il Way of Working, descrivendo ruoli, responsabilità, riunioni e modalità di collaborazione per garantire efficienza e qualità.

4.5 Verbali esterni

Questa cartella contiene i file pdf dei verbali delle riunioni con i rappresentanti dell'azienda proponente. Vengono documentati confronti, chiarimenti e decisioni concordate.

4.6 Verbali interni

Questa cartella contiene i file pdf dei verbali delle riunioni svolte tra i membri del gruppo. Vengono documentate decisioni prese, pianificazioni e attività interne.

4.7 docs

Questa cartella contiene tutti i file relativi al nostro sito web, in particolare:

- **assets**: cartella contenente tutte le immagini.
- **css**: cartella contenente il main.css, per lo stile del sito.
- **js**: cartella contenente i file .js.
- **files.html**: file html della sezione "tutti i file" del sito.
- **index.html**: file html della home del sito.

4.8 scripts

Questa cartella contiene degli scripts, quali uno script(gulpease.py) per calcolare l'indice di leggibilità di un testo, un correttore automatico (spellcheck.py) e infine un file whitelist.txt che contiene l'elenco di tutte le parole da ignorare per la correzione ortografica.

4.9 src

Questa cartella contiene i file sorgenti LaTeX, divisi nelle rispettive cartelle (omonime a quelle contenenti i pdf relativi).

4.10 README.md

Questo è il file che presenta il gruppo e la repository.

5 Pianificazione del lavoro

Il lavoro del team è organizzato in sprint, al termine dei quali avviene una rotazione dei ruoli. Questa pratica consente a tutti di acquisire esperienza in diverse aree e di migliorare la collaborazione complessiva del gruppo.

5.1 Divisione dei ruoli

Il gruppo adotta una rotazione dei ruoli per garantire:

- Apprendimento completo di tutti i membri
- Distribuzione equa delle responsabilità
- Sviluppo di competenze trasversali

5.2 Descrizione dei ruoli

Di seguito viene fornita una descrizione dettagliata dei diversi ruoli, illustrando in modo chiaro, completo e ben strutturato le responsabilità, i compiti specifici e le aree di competenza associate a ciascuna posizione. L'obiettivo è fornire una panoramica accurata che permetta di comprendere pienamente le funzioni svolte da ogni ruolo all'interno dell'organizzazione.

5.2.1 Responsabile

- Comunica con i professori e l'azienda (tramite email);
- Si occupa dell'approvazione finale di tutti i documenti e file;
- Redige i verbali;
- Si occupa dei diari di bordo.

5.2.2 Amministratore

- Sistema la repository: crea e assegna le issues, crea le milestone...;
- Aggiorna il sito web del gruppo;
- Si occupa della redazione dei documenti.

5.2.3 Analista

- Analisi dei requisiti e dei casi d'uso.

5.2.4 Progettista

- Progetta l'architettura del software;
- Sceglie le tecnologie da utilizzare;
- Scrive la documentazione di progetto (ad esempio i diagrammi UML);
- Aiuta i programmatori a implementare le scelte fatte.

5.2.5 Programmatore

- Scrive il codice.

5.2.6 Verificatore

- Controlla codice e documenti.

5.3 Sprint planning

- Durata sprint: 2 settimane
- Planning meeting all'inizio di ogni sprint
- Review meeting alla fine di ogni sprint
- Retrospettiva per il miglioramento continuo

6 Analisi dei Requisiti (AdR)

L'**analisi dei requisiti** rappresenta una fase fondamentale del processo di sviluppo, in quanto permette di identificare gli obiettivi del sistema, le funzionalità attese e i vincoli da rispettare. Tale attività costituisce il fondamento su cui vengono impostate le successive fasi di progettazione e implementazione. Il gruppo ha adottato un approccio collaborativo finalizzato a garantire una definizione adeguata dei requisiti.

6.1 Identificazione iniziale dei requisiti

La prima fase dell'analisi ha previsto un insieme di riunioni dedicate alla raccolta di tutti i possibili use case, utilizzando la tecnica del brainstorming. Durante questi incontri il team ha:

- discusso le esigenze principali del sistema;
- ipotizzato diversi casi d'uso e potenziali funzionalità;
- chiarito dubbi e definito una struttura condivisa per la descrizione dei requisiti e dei casi d'uso, così da evitare ambiguità.

Questa fase ha permesso di costruire una visione comune del dominio applicativo e di delineare una prima struttura dei casi d'uso principali.

6.2 Analisi individuale dei casi d'uso

Una volta definita la lista di tutti i casi d'uso possibili, ogni membro del gruppo ha assunto la responsabilità di analizzarne uno in maniera approfondita. In particolare, ognuno ha:

- studiato nel dettaglio il caso d'uso assegnato;
- definito attori, precondizioni, postcondizioni, scenario principale e eventuali estensioni;
- qualora necessario, studiato nel dettaglio anche eventuali sottocasi.

Questa suddivisione del lavoro ha permesso di svolgere un'analisi più accurata e di sfruttare in modo efficiente le competenze di ogni membro del gruppo.

6.3 Revisione dei casi d'uso

Una fase essenziale del processo è stata la revisione incrociata dei contenuti prodotti individualmente. Ogni membro ha esaminato il lavoro degli altri al fine di:

- verificare la coerenza rispetto alle decisioni emerse nelle riunioni iniziali;
- controllare l'aderenza agli standard interni stabiliti nella prima fase;
- individuare eventuali incoerenze, ridondanze o mancanze;
- migliorare la chiarezza e la completezza delle descrizioni.

Il confronto tra i membri del gruppo ha contribuito a uniformare lo stile, a garantire la qualità complessiva dell'analisi e a minimizzare il rischio di errori o interpretazioni divergenti.

6.4 Discussione dei casi d'uso con l'azienda

Il 2 dicembre 2025 il gruppo ha svolto anche una riunione dedicata all'analisi dei casi d'uso con l'azienda committente. Durante l'incontro sono stati presentati gli scenari elaborati dal team, con l'obiettivo di confrontare i casi d'uso individuati internamente e verificarne la correttezza rispetto alle reali esigenze del cliente. Questo confronto ha permesso di validare le ipotesi iniziali, correggere eventuali interpretazioni errate e allineare le aspettative del progetto tra gruppo e azienda.

6.5 Analisi individuale dei requisiti

Una volta definiti e validati i casi d'uso, il gruppo ha proceduto alla definizione dei requisiti, assegnando a ciascun membro la responsabilità di redigerne una parte. In particolare, ogni membro ha:

- scritto i requisiti partendo dai casi d'uso analizzati;
- formulato i requisiti in modo semplice ma chiaro;
- utilizzato una dicitura condivisa per identificare in modo univoco i requisiti, distinguendo tra requisiti obbligatori (O), desiderabili (D) e opzionali (OP);
- classificato ogni requisito in base alla sua tipologia, distinguendo tra requisiti funzionali (F), di qualità (Q) e di vincolo (V).

Questa organizzazione del lavoro e l'adozione di una notazione comune hanno permesso di ottenere un insieme di requisiti coerente, uniforme e facilmente comprensibile, permettendo inoltre la verifica e eventuale correzione ulteriore dei casi d'uso precedentemente scritti.

6.6 Revisione dei requisiti

Una fase essenziale del processo è stata la verifica incrociata dei requisiti redatti. I membri del gruppo si sono impegnati ad esaminare tutti i requisiti al fine di:

- verificare la coerenza con i casi d'uso precedentemente definiti;
- controllare la completezza dei requisiti rispetto alle funzionalità previste;
- individuare eventuali requisiti mancanti, ridondanti o poco chiari;
- migliorare la chiarezza e la precisione delle formulazioni dei requisiti.

Il confronto tra i membri del gruppo ha contribuito a rafforzare la qualità complessiva del documento dei requisiti e a ridurre il rischio di errori, mancanze o ridondanze.

6.7 Scrittura del documento definitivo

Al termine delle fasi precedenti, il team ha proceduto alla redazione del documento "Analisi dei Requisiti", nel quale tutti i casi d'uso sono stati integrati e uniformati in un'unica struttura coerente. Il risultato è un insieme di requisiti chiari, consistenti e verificabili.

6.8 Giudizio sull'approccio adottato

L'approccio adottato durante l'analisi dei requisiti ha contribuito in modo significativo alla crescita del team, sia sul piano tecnico sia su quello organizzativo. Grazie alle attività svolte, il gruppo ha:

- imparato a strutturare i casi d'uso in maniera chiara e coerente, anche dopo un iniziale disallineamento nello stile di documentazione che ha richiesto un successivo lavoro di uniformazione;
- acquisito maggiore familiarità con i processi di revisione incrociata, migliorando la capacità di individuare errori, ambiguità o parti mancanti nel lavoro dei colleghi;
- migliorato le abilità di collaborazione e comunicazione all'interno del gruppo;
- sviluppato una migliore capacità di analizzare i requisiti in modo critico.

7 Piano di Progetto (PdP)

Il **Piano di Progetto** costituisce uno strumento fondamentale per organizzare, monitorare e coordinare le attività del team durante l'intero ciclo di sviluppo. Esso definisce la pianificazione temporale, la distribuzione dei ruoli, il carico di lavoro dei membri tramite un riassunto di ogni sprint e l'analisi dei potenziali rischi, garantendo una gestione strutturata e consapevole del progetto.

7.1 Analisi dei rischi

Un elemento centrale del Piano di Progetto è stata l'analisi dei rischi, svolta con l'obiettivo di identificare gli eventi critici che avrebbero potuto compromettere tempi, qualità o continuità del lavoro. L'analisi è stata condotta esaminando individualmente ciascun potenziale problema, classificandolo all'interno di tre categorie principali: rischi organizzativi, rischi tecnologici e rischi legati al prodotto. Per ogni rischio sono stati definiti gli elementi essenziali per una valutazione completa:

- una descrizione chiara del problema;
- la probabilità che succedano;
- la gravità del problema;
- le misure di prevenzione;
- le misure di mitigazione.

In questo modo abbiamo potuto capire meglio i possibili problemi e prepararci a gestirli.

7.2 Approccio adottato, identificazione dei ruoli e pianificazione delle attività

Il nostro approccio usa la metodologia Agile, in particolare il framework Scrum, che prevede una pianificazione continua e iterativa del lavoro. Fin dall'inizio sono stati definiti i ruoli di progetto, assegnati e alternati tra i membri del team nei diversi Sprint per garantire una distribuzione equilibrata delle responsabilità e favorire la crescita delle competenze. La pianificazione delle attività è stata gestita Sprint per Sprint, adattandosi ai progressi del team e ai risultati ottenuti.

7.3 Riassunto di ciascuno Sprint

A conclusione di ogni Sprint, il gruppo ha redatto un resoconto delle attività svolte, documentando le sue fasi:

- Pianificazione: con obiettivi, ruoli, rischi e ore occupate previsti;
- Review: con attività e ore svolte e eventuali problemi riscontrati;
- Retrospettiva: con eventuali dubbi o problemi e la soluzione trovata, e eventuali attività programmate per lo sprint successivo.

8 Piano di Qualifica (PdQ)

Il **Piano di Qualifica** è il documento che definisce la strategia del gruppo per garantire la qualità del prodotto software e l'efficienza dei processi di sviluppo. Rappresenta il riferimento principale per tutte le attività di Verifica e Validazione.

8.1 Scopo del documento

- **Pianificazione:** Fissare *ex-ante* gli standard qualitativi da raggiungere. Definisce quali metriche utilizzare, come calcolarle e quali sono le soglie di accettabilità (minime) e di ottimalità (desiderabili).
- **Consuntivo (Cruscotto):** Riportare *ex-post* i risultati delle misurazioni effettuate durante le varie fasi di progetto, permettendo di valutare oggettivamente lo stato di salute del progetto e la conformità del prodotto rispetto ai requisiti.

8.2 Contenuto e Struttura

Il documento è strutturato per rispondere alle norme di qualità ISO/IEC 9126 e ISO/IEC 25010 ed è suddiviso nelle seguenti macro-aree:

- **Qualità di Processo:** Definisce le metriche per monitorare l'andamento dei lavori, inclusi la gestione del budget, la pianificazione temporale e la stabilità dei requisiti.
- **Qualità di Prodotto:** Definisce le metriche per valutare il software prodotto, coprendo aspetti quali l'adeguatezza funzionale, l'efficienza, l'usabilità, l'affidabilità e la manutenibilità.
- **Strategia di Verifica e Testing:** Descrive le procedure operative per l'Analisi Statica e l'Analisi Dinamica (livelli di test: Unit, Integration, System, Acceptance).
- **Cruscotto di Qualità:** Una sezione dinamica che viene aggiornata ad ogni revisione di progetto, contenente i grafici e le tabelle con i valori effettivi misurati per ogni metrica, evidenziando eventuali non conformità e le relative azioni correttive intraprese.

8.3 Metodologia di calcolo dell'Earned Value (EV)

Al fine di monitorare oggettivamente l'avanzamento del progetto ed evitare stime soggettive, il gruppo adotta la tecnica di misurazione fisica 0/100 (All-or-Nothing). Il calcolo avviene secondo le seguenti regole:

1. Ogni attività (Task/Issue) pianificata nello Sprint ha un valore in ore associato, definito nel Piano di Progetto.
2. Al termine dello Sprint, o nei momenti di verifica intermedi, si controlla lo stato di ciascuna attività.
3. Il valore dell'attività viene sommato all'Earned Value (EV) totale se e solo se l'attività soddisfa interamente la Definition of Done (DoD).
4. Se un'attività è parzialmente completa (es. codice scritto ma non revisionato o verificato), il suo contributo all'EV è 0.

8.4 Definition of Done (DoD)

Affinché un'unità di lavoro (Task/Issue) possa essere considerata "Fatta" (Done) e conteggiata nel calcolo dell'Earned Value, deve soddisfare i seguenti criteri:

- **Codice:** Il codice sorgente è stato scritto, committato e pushato sul repository ufficiale nel branch corretto.
- **Documentazione:** Il codice è corredato di commenti ed eventuali modifiche o scelte tecniche sono state riportate nella documentazione ufficiale.
- **Analisi Statica:** Il codice supera i controlli di stile e qualità senza errori o warning bloccanti.
- **Revisione:** Il lavoro svolto è stato revisionato e verificato da uno o più membri del gruppo.

8.5 Strumenti e Metriche di Qualità Automatizzate

Al fine di garantire un monitoraggio continuo e oggettivo della qualità, il gruppo ha integrato nel processo di sviluppo una serie di script automatici eseguiti tramite GitHub Actions. Di seguito vengono descritte le metriche monitorate e il funzionamento degli strumenti adottati.

8.5.1 Qualità della Documentazione

Correttezza Ortografica (CO)

- **Descrizione e Scopo:** Misura il numero di errori ortografici di battitura presenti nei documenti. L'obiettivo è garantire professionalità e assenza di refusi che potrebbero compromettere la comprensione del testo.
- **Calcolo:** Conteggio ed esposizione delle parole non presenti nel dizionario italiano standard e non incluse nella `whitelist` di progetto (che contiene acronimi, nomi propri e termini tecnici accettati).
- **Automazione:** Uno script Python (`scripts/spellcheck.py`) analizza ricorsivamente tutti i file `.tex`. Lo script rimuove i comandi LaTeX (es. `\section`, `\textbf`) per analizzare solo il testo puro. Le parole sconosciute vengono confrontate con la libreria `pyspellchecker` e, se non presenti nella `whitelist`, vengono segnalate come errori. Le parole individuate vanno controllate e corrette nei documenti per evitare errori ortografici.

Indice di Gulpease (IG)

- **Descrizione e Scopo:** È un indice di leggibilità calibrato sulla lingua italiana. Valuta quanto un testo sia comprensibile in base alla lunghezza delle parole e delle frasi.
- **Calcolo:** La formula utilizzata è:

$$89 + \frac{300 \times (\text{numero frasi}) - 10 \times (\text{numero lettere})}{\text{numero parole}}$$

- **Automazione:** Uno script Python (`scripts/gulpease.py`) analizza i file sorgente, calcolando i totali di lettere, parole e frasi (riconoscendo la punteggiatura di fine periodo). Restituisce un valore medio per documento, che deve essere superiore a 40. Se tutti i documenti hanno un IG adeguato, il badge presente nel README della repo della documentazione diventa verde; in caso contrario diventa rosso e i file vanno rivisitati.

8.5.2 Qualità del Codice

Lines of Code (LOC)

- **Descrizione e Scopo:** Rappresenta la dimensione fisica del software misurata in righe di codice. Serve a monitorare la crescita del progetto e a stimare lo sforzo di manutenzione.
- **Calcolo:** Conteggio delle righe non vuote e non di commento nei file sorgente.
- **Automazione:** Viene utilizzato il tool `cloc` (Count Lines of Code) all'interno della pipeline GitHub. Lo strumento è configurato per escludere file generati automaticamente, immagini, librerie esterne (es. `node_modules`) e file di configurazione, fornendo un conteggio fedele del lavoro svolto dal team. Questa metrica non è vincolante ma permette di monitorare la dimensione dei sorgenti.

Complessità Ciclomatica (McCabe)

- **Descrizione e Scopo:** Misura la complessità logica di una funzione o metodo contando il numero di cammini linearmente indipendenti attraverso il codice sorgente. Una complessità elevata indica codice difficile da testare e mantenere.
- **Calcolo:** Basato sul grafo di flusso di controllo del programma. Semplificando, incrementa il conteggio per ogni struttura di controllo (`if`, `while`, `for`, `case`).
- **Automazione:** Viene utilizzato il tool `lizard`. La pipeline analizza i file di logica (JavaScript, PHP, Python) ignorando i file puramente strutturali o di stile (HTML, CSS, LaTeX). Il tool segnala la complessità media (CCN) e avvisa se specifiche funzioni superano la soglia di guardia stabilita nel Piano di Qualifica.

8.5.3 Visualizzazione dei Risultati

L'esecuzione di questi controlli avviene automaticamente ad ogni **push** sulla repository remota. Per consultare i risultati:

1. Accedere alla scheda **Actions** della repository GitHub.
2. Selezionare l'ultimo workflow eseguito (es. *"Compila LaTeX e Verifica Qualità"* o *"Code Metrics"*).
3. Cliccare sui singoli job (es. *"Check Spelling"*, *"Calcolo LOC"*) per espandere il log e visualizzare i report dettagliati e le tabelle riassuntive generate dagli script.
4. Le metriche per la qualità della documentazione sono calcolate nella repository della documentazione ufficiale del gruppo.
5. Le metriche per la qualità del codice sono calcolate nella repository del POC e del prodotto finale.